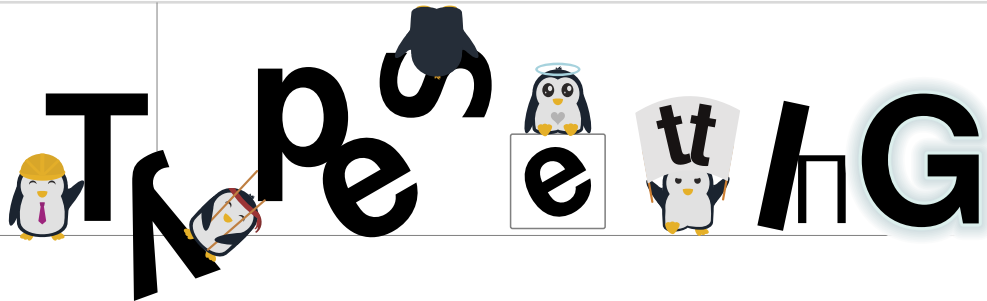


# An introduction to $\text{\LaTeX} 2_{\epsilon}$

Typesetting Articles, Reports, and Presentations

Florian Sihler (Ulm University) | September 20, 2026



# Topic Overview

## 1. Getting Started

## 2. Basic Formatting

- Commands
- Shapes, Series, and Spacing
- Sections and Lists
- Colors and Packages

## 3. Math

- Simple Formulas
- Advanced Math Typesetting

## 4. Figures & Tables

- Including Images
- Placing Images
- Typesetting Tables

## 5. Listings

- The `listings` Package
- The `minted` Package

## 6. Hyperlinks

- Creating Titles
- The `hyperref` Package
- Including Metadata

## 7. More

- Bibliographies
- Making Presentations
- TikZ — The Behemoth
- Changing the Document
- Custom Macros and More

# Getting started

- You do not have to install anything to get started
- You can use Overleaf: <https://www.overleaf.com>
- In the long run, however, we recommend an installed version of  $\text{\LaTeX}$ 
  - Linux already has a package available (apt: `sudo apt install texlive-full`)  
Otherwise: <https://www.tug.org/texlive/tlmgr.html>
  - For Windows you can use  $\text{MiKTeX}$  in combination with an IDE like  $\text{\TeXstudio}$
  - For MacOS there are great tutorials
  - Even for iPad there are Apps like  $\text{\TeXpad}$



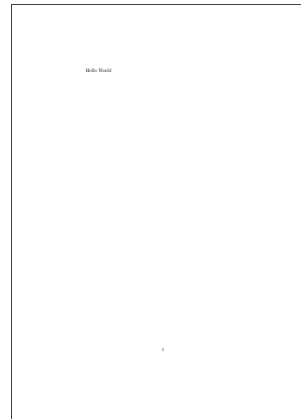
# A simple “Hello World!”

Compile by pressing the corresponding button  
or use **pdf~~l~~atex**/**l**atexmk from the commandline.

```

1 \documentclass{article}
2
3 \begin{document}
4   Hello World
5 \end{document}

```



# A simple “Hello World!”

```

1 \documentclass{article}
2
3 \begin{document}
4   Hello World
5 \end{document}

```

Hello World

*From now on, examples will be cropped automatically.*

# Structure of a document

*The document class defines the basic layout of a document*

*For this example, we use article.*

```
1 \documentclass{article}
```

```
2
```

```
3 \begin{document}
```

```
4     Hello World
```

```
5 \end{document}
```

*The Preamble: Used to load packages and commands*

*The main document*

# What is a “Command”?

- $\text{\LaTeX}$ -commands start with a backslash ( $\backslash$ )

```
1 \documentclass{article}
2 \begin{document}
3   Hello World
4 \end{document}
```

- Arguments are passed in curly braces (e.g., `article`)
- (Line-)Comments start with a percent sign (%)

# First formatting commands

- To write in “**boldface**”, “*italic*”, or “**teletype**”, use the corresponding commands:

```

1 \documentclass{article}
2
3 \begin{document}
4 Hello, \textbf{just \textit{great}} \texttt{format}.
5 \end{document}

```

Hello, just *great* format.

- As you can see, these commands can be nested (as long as the font supports this)
- Later, we will define our own commands for this (e.g., `\algorithm{x}` instead of `\textbf{\textit{x}}`)

# A simple document

- Together with `\\` which forces a new line we can create a simple document:

```

1 \documentclass{article}
2
3 \begin{document}
4 \textbf{1. Things I like} \\
5 - Penguins \\- Ducks \\
6
7 \textbf{2. Things I do not like}\\
8 Word, PowerPoint, and sentences deliberately ←
   stretched so they force a \textit{line break}.
9 \end{document}

```

1. Things I like  
- Penguins  
- Ducks

2. Things I do not like  
Word, PowerPoint, and sentences deliberately stretched so they force a *line break*.

- However, please be aware that you should not force line breaks “real” document (there are better ways)
- Furthermore, you should refrain from using low-level markup-commands like `\textbf` directly

# The importance of new lines

- A simple line break will be transformed to a space
- Multiple spaces collapse to a single space
- An empty line starts a new paragraph (`\par`) which is indented by default (by `\parindent`)
- Multiple empty lines will be collapsed to a single empty line

```

1 \documentclass{article}
2
3 \begin{document}
4 Hello World
5
6 Amazing! Sentence
7 \end{document}

```

Hello World  
Amazing! Sentence

# Sections

- Labeling your segments with `\textbf{1. My Name}` is very inconvenient
- For this,  $\text{\LaTeX}$  provides `\section`, `\subsection`, and more (depending on the document class)
- This allows you to create a table of contents as well!

```

1 \documentclass{article}
2
3 \begin{document}
4 \tableofcontents
5 \section{Things I like}
6 - Penguins \- Ducks
7
8 \section{Things I do not like}
9 \subsection{Word!}
10 Word, PowerPoint,...
11 \end{document}

```

## Contents

### 1 Things I like

- Penguins  
- Ducks

### 2 Things I do not like

#### 2.1 Word!

Word, PowerPoint,...

# How $\text{\LaTeX}$ produces the table of contents

- When reading your document,  $\text{\LaTeX}$  collects all `\section`, `\subsection`, ... commands and writes them to a separate file
- If  $\text{\LaTeX}$  is run again it can use the information in this file (ending in `.toc`) to typeset the table of contents
- If you compile your document manually, you require a second run to update the table of contents
- Tools like `latexmk` or an IDE automates this for you

# Items and enumerations

- Similar to sections,  $\text{\LaTeX}$  provides nicer ways to create lists
- For unordered lists, it offers the `itemize`, for ordered lists the `enumerate` environment
- Let's look at an example:

```

1 \documentclass{article}
2
3 \begin{document}
4 \section{Things I like}
5 \begin{itemize}
6   \item Penguins
7   \item Ducks
8 \end{itemize}
9 \end{document}

```

## 1 Things I like

- Penguins
- Ducks

# Colors

- In order to use colors in  $\LaTeX$ , we want to use our first package!
- For this, we will use `xcolor` and load it with `\usepackage{xcolor}`
- We can use predefined colors (`\textcolor{<color>}{x}`), or define our own!

```

1 \documentclass{article}
2 \usepackage{xcolor}
3 \definecolor{myOwnBlue}{RGB}{0, 100, 224}
4
5 \begin{document}
6 \textcolor{orange}{Orange Text!}
7 \textcolor{myOwnBlue}{Blue Text!}
8 \end{document}

```

Orange Text! Blue Text!

# A default preamble

- Besides `xcolor`, there are several packages useful in every  $\text{\LaTeX}$ -Document
  - `inputenc` and `fontenc` to ensure consistent encodings
  - `babel` for advanced language support (hyphenation, ...)
  - `microtype` for improved micro typography
- Some of them take (optional) arguments in square brackets

```

1 \documentclass{article}
2 \usepackage[T1]{fontenc}
3 \usepackage[utf8]{inputenc}
4 \usepackage[english]{babel}
5 \usepackage{microtype}
6
7 \begin{document}
8 Hello!
9 \end{document}

```

Hello!

# Comments regarding your preamble

- With time, you will build your own preamble
- While several packages have a lot of package-options and some of them have interactions, you learn them rather quickly and on a need-to-use basis
- For example, if you want to configure babel for a german text, use ngerman (“new-german”)

# Typesetting formula

- $\text{\LaTeX}$  offers two modes to typeset a formula (we are interested in):  
 Inline: with `\(a + b\)` (to be used within text)  
 Displaystyle: with `\[a + b\]` (larger, centers the formula as well)
- Spaces are treated differently, as  $\text{\LaTeX}$  will automatically apply spacing rules

```

1 \documentclass{article}
2
3 \begin{document}
4 \begin{itemize}
5   \item 42 + 3=45
6   \item \((42 + 3=45\))
7   \item \(\displaystyle 42 + 3=45\)
8   \item \[42 + 3=45\]
9 \end{itemize}
10 \end{document}
```

- 42 + 3=45
- 42 + 3 = 45
- 42 + 3 = 45
- 

42 + 3 = 45

# Special symbols

- There are a lot of special symbols you can use
  - `\alpha`, `\beta`, `\gamma`, ... for greek symbols
  - `\sin`, `\sum`, `\prod`, `\int` (integral) for common functions & operations
  - `\infty` (infinity), `\neq` (not equal), ... for other common symbols
- Although they are intuitive, you can use [detexify.kirelabs.org](https://detexify.kirelabs.org) to draw the symbol you want and get the corresponding  $\text{\LaTeX}$ -command!
- You can use `^{\{X\}}` and `_{\{Y\}}` to write (even nested) super- and subscripts



# Some simple formulas

- Let's combine our math-knowledge (using `\frac` to typeset a fraction):

```

1 \documentclass{article}
2
3 \begin{document}
4 When calculating \(\sum_{i = 1}^{\infty} \frac{2}{i^2} \neq 0\) one must be careful!
5 You are advised to eat a piece of \(\pi\) first or calculate \(\sin(\pi)\).
6 Is it \(1\), \(-1\), \(\pm 1\), or even \(\geq 1\)?
7 \end{document}

```

When calculating  $\sum_{i=1}^{\infty} \frac{2}{i^2} \neq 0$  one must be careful! You are advised to eat a piece of  $\pi$  first or calculate  $\sin(\pi)$ . Is it 1,  $-1$ ,  $\pm 1$ , or even  $\geq 1$ ?

# The mathtools package

- The `mathtools` package offers a lot of useful features for math typesetting
- Together with it, we want to learn about two important environments:
  - equation: For a single, numbered formula (you can reference)
  - align: For aligning multiple formulas using the `&`-character
- Some useful notes:
  - Within formulas you can use the `\text`-command to write “normal text”
  - We can label equations with `\label{x}` and reference them with `\ref{x}` in the text
  - Some characters (like `&` or `#`) are reserved by  $\text{\LaTeX}$ . Escape them with a preceding backslash (e.g., `\&`)
  - The special tilde `~` can be used to insert a non-breaking space (e.g., useful for references)

# Advanced math example

```

1 \documentclass{article}
2 \usepackage{mathtools}
3
4 \begin{document}
5 My formula~\ref{eq:example}:
6 \begin{equation}
7   \frac{n!}{a! \cdot (n-a)!} = \binom{n}{a}
8   \label{eq:example}
9 \end{equation}
10 Another one:
11 \begin{align}
12   f(0) &= \sqrt{1} \quad \label{eq:example2} \\
13   f(n) &= \begin{cases}
14     f(n-2) & \text{if } n \bmod 2 = 0 \\
15     f(n-1) - f(n-2) & \text{otherwise}
16   \end{cases}
17 \end{align}
18 \end{document}

```

My formula 1:

$$\frac{n!}{a! \cdot (n-a)!} = \binom{n}{a} \quad (1)$$

Another one:

$$f(0) = \sqrt{1} \quad (2)$$

$$f(n) = \begin{cases} f(n-2) & \text{if } n \bmod 2 = 0 \\ f(n-1) - f(n-2) & \text{otherwise} \end{cases} \quad (3)$$

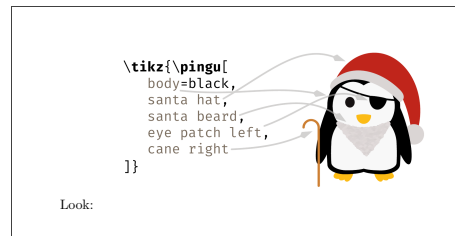
# Let's include an image

- In order to load external graphics, we use the `graphicx` package
- With it, we get the `\includegraphics`-command which loads an image based on its path

```

1 \documentclass{article}
2 \usepackage{graphicx}
3
4 \begin{document}
5   Look:
6   \includegraphics{images/santa-pingu.png}
7 \end{document}

```



- While you do not need to put the image in a separate folder, we recommend you doing so to keep your project organized

# Transform images

- You can pass several options to the `\includegraphics`-command
  - width & height: To resize the image to a specific width or height
  - scale: To scale the image by a given factor
  - keepaspectratio: Keep the aspect ratio of the image
  - angle: Rotate the image by a given angle

```

1 \documentclass{article}
2 \usepackage{graphicx}
3
4 \begin{document}
5   \includegraphics[width=7.5cm]
6     {images/santa-pingu.png}
7   \includegraphics[scale=0.25, angle=30]
8     {images/santa-pingu.png}
9 \end{document}

```



# Useful notes on images

- Make sure your images have a sufficient resolution
- The command `\linewidth` holds the width of the current line
- `\textwidth` holds the full width (e.g., both columns in a multi-column document)
- You can prefix them with a factor (like `0.5\linewidth`)
- Otherwise, there are several units at your disposal (mm, cm, pt, ...)  
See <https://github.com/tweh/tex-units> for a pretty overview



# Placing Images

- Normally you do not want figures to be placed in the middle of a paragraph but keep the text together
- Therefore,  $\text{\LaTeX}$  provides the `figure` environment (which allows for a `\caption`)
- The example uses the `blindtext` package to provide dummy text

```

1 \documentclass{article}
2 \usepackage{blindtext,graphicx}
3
4 \begin{document}
5 \blindtext
6
7 \begin{figure}
8 \centering
9 \includegraphics{images/santa-pingu.png}
10 \caption{Santa Penguin}
11 \end{figure}
12
13 \blindtext
14 \end{document}

```



# Referencing figures

- Similar to equations you can reference figures with `\label` and `\ref`
  - Besides `\ref`, there are other commands like `\pageref` which give you the page number
  - A package like `hyperref` allows for much more (see page 41)
- Make sure, that `\label` appears *after* the caption!

```

1 \documentclass{article}
2 \usepackage{graphicx}
3
4 \begin{document}
5 \begin{figure}
6 \centering \includegraphics{images/santa-pingu.png}
7 \caption{Santa Pinguin}
8 \label{fig:santa-pingu}
9 \end{figure}
10
11 Let's have a look at Figure~\ref{fig:santa-pingu}
12 on page~\pageref{fig:santa-pingu}.
13 \end{document}

```

```

\tikz{\pingu[
  body=black,
  santa hat,
  santa beard,
  eye patch left,
  cane right
]}

```



Figure 1: Santa Pinguin

Let's have a look at Figure ?? on page ??.

# Fine-tune figure placement

- You can specify the positions that  $\text{\LaTeX}$  tries to place the “figure-float”
  - top**: allow to place at the top of the page
  - bottom**: allow to place at the bottom of the page
  - here**: allow to place at position in document
  - page**: allow to place on its own page
- There are others, like
  - `H` to force the *here* placement (from the `float` package)
  - `\FloatBarrier` to limit the float position (from the `placeins` package)
- You can specify them in any combination (the order does not matter)
- Let the ~~force~~ float guide you (do not overdo it with forced placements,  $\text{\LaTeX}$  is usually pretty good)

# Example for figure placement

```

1 \documentclass{article}
2 \usepackage{blindtext,graphicx}
3
4 \begin{document}
5 \blindtext
6
7 \begin{figure}[hb]
8 \centering
9 \includegraphics{images/santa-pingu.png}
10 \caption{Santa Penguin}
11 \end{figure}
12
13 \blindtext
14 \end{document}

```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.



Figure 1: Santa Penguin

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis

# There is a lot more

- You can let text go around your floats with the `wrapfig2` package
- Captions can be controlled with the `caption` package
- Subfigures can be obtained by using the `subcaption` package
- You can create your own floats (like figures) with the `newfloat` package
- If you want a list of all figures, you can use the `\listoffigures` command (similar to `\tableofcontents`)

# Typesetting a simple table

- Let's start with an example:

```

1 \documentclass{article}
2
3 \begin{document}
4 \begin{tabular}{|l|c|r|}
5   \hline
6   \textbf{Important} & Columns & Look & Good \\
7   \hline \hline
8   1                & 2      & 3    & 4    \\
9   5                & 6      & 7    & 8    \\
10  \hline
11 \end{tabular}
12 \end{document}

```

| Important | Columns | Look | Good |
|-----------|---------|------|------|
| 1         | 2       | 3    | 4    |
| 5         | 6       | 7    | 8    |

- The `tabular` environment takes the desired column specification as its first argument
  - Alignments for (left, center, and right)
  - Vertical lines with `|`, horizontal lines with `\hline`
  - Use `&` to separate columns and `\\` to start a new row

# Improving your tabular

- The `booktabs` package improves the look of your tabular
  - It provides commands like `\toprule`, `\midrule`, and `\bottomrule` to replace `\hline`
- `array` improves on the `tabular` environment and provides a lot more
  - You can specify custom columns, and a prefix/suffix for cells
  - Get centered columns with a fixed width (e.g. `m{2cm}`)
- Similar to the `figure`, there is the `table` float for your tables

# Example for our new table

```

1 \documentclass{article}
2 \usepackage{blindtext,array,booktabs}
3
4 \begin{document}
5 \begin{table}
6 \centering
7 \begin{tabular}{l c m{1cm} r}
8 \toprule
9 \textbf{Important} & Columns & Look & Good \\
10 \midrule
11 1 & 2 & pretty long for an one cm column & 4 \\
12 \bottomrule
13 \end{tabular}
14 \caption{My Great Table}
15 \label{tbl:mytable}
16 \end{table}
17 Look at page~\pageref{tbl:mytable}.
18 \blindtext
19 \end{document}

```

| Important | Columns | Look                             | Good |
|-----------|---------|----------------------------------|------|
| 1         | 2       | pretty long for an one cm column | 4    |

Table 1: My Great Table

# Again, there is so much more

- The `tabularx` package allows for automatic column widths with its `X` column
- Besides amazing support for typesetting units, the `siunitx` provides the `S` column to align numbers at will
- You can use the `longtable` package to typeset tables that span multiple pages
- Although the `tabu` package is not as steadily maintained, it still offers a lot of great things (like alternating row colors)
- If you dislike the ways to specify tables, you can use an online generator like <https://www.tablesgenerator.com/>



# Typesetting code with the `listings` package

- As some characters have a special meaning in  $\text{\LaTeX}$  ( $\backslash$ ,  $\#$ , spaces ...), we cannot just “paste” source code
- With the `listings` package however, we can typeset source code in a very flexible way!
- Let’s look at an example with the `lstlisting` environment:

```

1 \documentclass{article}
2 \usepackage{listings}
3
4 \begin{document}
5 \begin{lstlisting}[language=java]
6 public static void main(String[] args) {
7     System.out.println("Hello World!");
8 }
9 \end{lstlisting}
10 \end{document}

```

```

public static void main(String[] args) {
    System.out.println("Hello World!");
}

```

# Changing the style of the code

- To get the common teletype-font, we can change the `basicstyle` to use the `\ttfamily`
- As several options will remain the same for all (or at least most) listings, we can set them with `\lstset{<options>}`
- Similarly, we can make use of colors

```

1 \documentclass{article}
2 \usepackage{xcolor,listings}
3 \lstset{
4   basicstyle=\ttfamily,
5   stringstyle=\color{teal}
6 }
7
8 \begin{document}
9 \begin{lstlisting}[language=java]
10 public static void main(String[] args) {
11   System.out.println("Hello World!");
12 }
13 \end{lstlisting}
14 \end{document}

```

```

public static void main(String[] args) {
    System.out.println("Hello World!");
}

```

# Changing the font

- Sadly, the default teletypefont does not come with a boldface variant
  - We can change it by simply loading a package (e.g., `beramono`)
  - Be sure to set the font-encoding correctly (e.g., `T1`)
- For a huge list of font-packages see the [L<sup>A</sup>T<sub>E</sub>X font catalogue](#)

```

1 \documentclass{article}
2 \usepackage[T1]{fontenc}
3 \usepackage{xcolor,listings}
4 \usepackage[scaled]{beramono}
5 \lstset{basicstyle=\ttfamily,
6   stringstyle=\color{teal}}
7
8 \begin{document}
9 \begin{lstlisting}[language=java]
10 public static void main(String[] args) {
11   System.out.println("Hello World!");
12 }
13 \end{lstlisting}
14 \end{document}

```

```

public static void main(String[] args) {
    System.out.println("Hello World!");
}

```



# Where to look for more

- The [documentation](#) gives a great overview of the packages' capabilities
  - You can add your own language with `\lstdefinlanguage`
  - You can create your own code-environment (e.g., with defaults) using `\lstnewenvironment`
  - You can create a letter shortcut with `\lstMakeShortInline`
  - With `numbers=left` you can add line numbers on the left side
  - `showstringspaces=false` removes the “\_” symbols within strings
  - All code shown in this presentation is typeset with this package
  
- If you want to typeset umlauts or perform replacements, have a look at [literates](#) and [wikibooks](#)

# The minted package

- The `minted` uses `pygments` to highlight source code
- As this package requires to call an external program (which has to be installed for that), you need to enable `--shell-escape` (if you use an IDE you may have to search for the option)
- Besides that, the basic workflow is rather similar

```

1 \documentclass{article}
2 \usepackage{minted}
3
4 \begin{document}
5 \begin{minted}{java}
6 public static void main(String[] args) {
7     System.out.println("Hello World!");
8 }
9 \end{minted}
10 \end{document}

```

<MINTED>

## minted vs. listings

- While minted has more powerful presets (by using pygments), it is way harder to customize
- Especially if you want to insert arbitrary markers or perform replacements, listings allows this far simpler
- Yet, play with both and use the one that fits your needs best  
(the author of this introduction prefers listings)

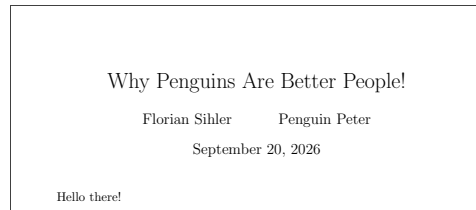
# Simple titles

- Within your document, you can use `\maketitle` to create a title(formatted by the set documentclass)
- Within the preamble, you can use the commands `\title`, `\author`, and `\date` to configure the title-information

```

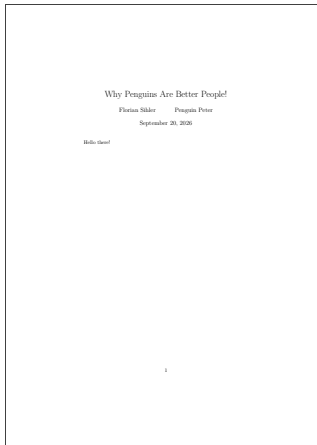
1 \documentclass{article}
2 \title{Why Penguins Are Better People!}
3 \author{Florian Sihler\and Penguin Peter}
4 \date{\today}
5
6 \begin{document}
7 \maketitle
8 Hello there!
9 \end{document}

```

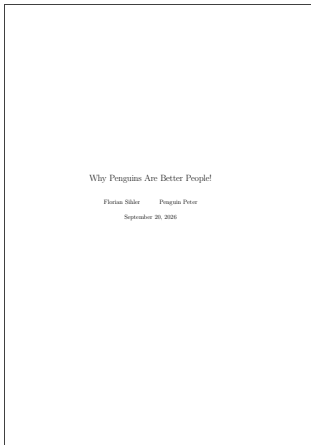


# How the document class changes the title-style

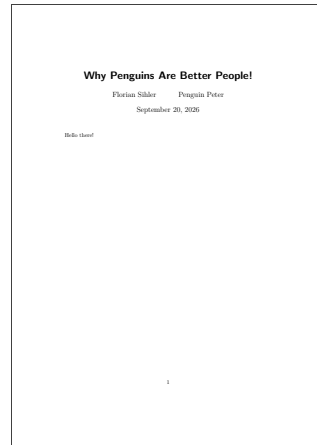
`\documentclass{article}`



`\documentclass{book}`



`\documentclass{scrartcl}`



Getting Started  
○○○

Basic Formatting  
○○○○○○○○○○

Math  
○○○○○

Figures & Tables  
○○○○○○○○○○○○

Listings  
○○○○○○

Hyperlinks  
○●○○○

More  
○○○○○○○○○○

# The `hyperref` package

- We already created some references (with `\tableofcontents`, `\ref`, ...)
- The `hyperref` package automatically transforms these references into clickable hyperlinks
- Furthermore it provides useful commands
  - `\autoref` is similar to `\ref` but tries to identify its target (Figure, ...) and adds the appropriate prefix
  - `\nameref` (due to the `nameref` package) prints the name of the target (e.g. section name, figure caption, ...)
  - `\hypersetup` controls the color of links, ...
- It is advised, to load the `hyperref` package after (almost) all other so it can modify their commands
- The `cleveref` and `varioref` packages are interesting as well!
- All links within this document are done with `hyperref` (and underlined with the help of `latex-underline`)

# A hyper document!

```

1 \documentclass{article}
2 \usepackage{xcolor,hyperref}
3 \hypersetup{colorlinks=true, allcolors=orange}
4
5 \begin{document}
6 \tableofcontents
7
8 \section{Introduction}\label{sec:intro}
9
10 \subsection{Why does everybody love penguins?}
11 Here, we are in \autoref{sec:intro},
12 to be found on page~\pageref{sec:intro}.
13 The name is \nameref{sec:intro}.
14 \end{document}

```

## Contents

### 1 Introduction

#### 1.1 Why does everybody love penguins?

Here, we are in ??, to be found on page ??. The name is ??.

*To avoid problems, I disabled the links within  
this preview and only activated their color.*

# Including metadata

- You can use `hyperref` to embed metadata into your document
- This is usually done for the title, author, ...
- You can use `\hypersetup` to configure the metadata with `pdftitle` and `pdfauthor`
- Alternatively the package-option `pdfusetitle` detects these automatically from the `\title` and `\author` commands

**There is so much more!**

# Bibliographies

- Citing is important, especially for scientific articles
- For this, have a look at the `bibtex` package (you have to run `biber`)

```

1 \documentclass{article}
2 \usepackage[backend=biber]{bibtex}
3
4 \addbibresource{data/example.bib}
5
6 \begin{document}
7 Hello World~\cite{Breckel2021}.
8 \printbibliography
9 \end{document}

```

Hello World [1].

## References

- [1] Alexander Breckel et al. “A domain-specific language for modeling and analyzing solution spaces for technology roadmapping”. In: *Journal of Systems and Software* 184 (2021), p. 111094. ISSN: 0164-1212. DOI: 10.1016/j.jss.2021.111094.

- There are tools to create and manage these bibliographies (Zotero, JabRef, ...)
- These reference work with `hyperref` (and there is so much more)

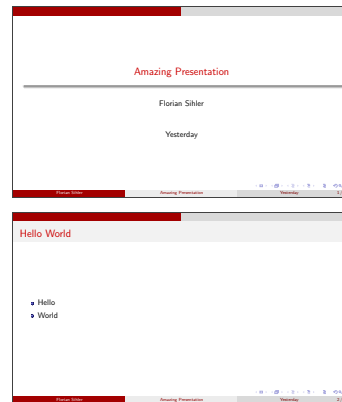
# Presentations — with the `beamer` document class

- The document you are seeing is written in  $\text{\LaTeX}$  using the `beamer`-based template from the `SDQ`.
- `beamer` is really powerful and allows for animations, multiple document versions, and more

```

1 \documentclass[aspectratio=169]{beamer}
2 \usetheme{CambridgeUS}
3 \title{Amazing Presentation}
4 \author{Florian Sihler}
5 \date{Yesterday}
6
7 \begin{document}
8 \maketitle
9 \begin{frame}{Hello World}
10   \begin{itemize}
11     \item Hello
12     \item World
13   \end{itemize}
14 \end{frame}
15 \end{document}

```



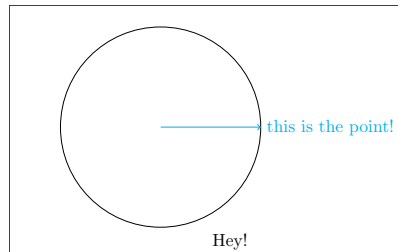
# TikZ — the behemoth

- How to draw anything? `tikz` is the way to go
  - This monster of a package allows a lot of drawing in  $\text{\LaTeX}$  it has an incredible learning curve
  - Want to plot something? use `pgfplots`
  - Want boxes? use `tcolorbox`

```

1 \documentclass{article}
2 \usepackage{tikz}
3
4 \begin{document}
5 \begin{tikzpicture}
6   \draw (0,0) circle[radius=2cm];
7   \draw[cyan,->] (0,0) -- (2,0)
8     node[right] {this is the point!};
9   \node[below] at (current bounding box.south) {Hey!};
10 \end{tikzpicture}
11 \end{document}

```



# Even the penguins!

- All the little penguins () you see are made with `tikz`
- Interested? See the `tikzpingus`
- Actively developed on [GitHub](#) (by me, the author)



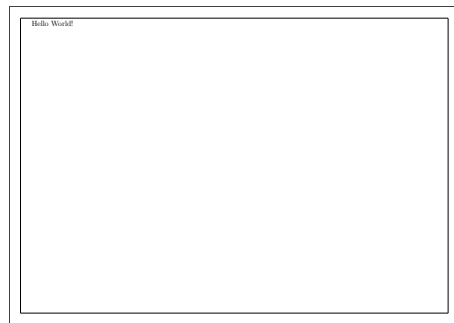
# The geometry package

- If you want to change the margins, the paper size, or the page layout, you can use the geometry package

```

1 \documentclass{article}
2 \usepackage[margin=5mm,paper=a5paper,
3     landscape,showframe]{geometry}
4
5 \begin{document}
6 Hello World!
7 \end{document}

```



# The parskip package

- Do you dislike the indentation performed at each paragraph (`\parindent`)?
- Consider the `parskip` package which exchanges the horizontal indentation for a vertical one

```

1 \documentclass{article}
2 \usepackage{parskip}
3
4 \begin{document}
5 This is a paragraph!
6
7 And this is another one!
8 No indentation, just vertical space!
9 \end{document}

```

This is a paragraph!

And this is another one! No indentation, just vertical space!

# Some general notes

- You can learn most things on a need-to-have basis
  - *Want quotes?*  
Use the `csquotes` package and its `\enquote` command
  - *Dislike the name of a command?*  
The `\let` command allows you to create a static alias for most commands (`\let\say\enquote`)
  - *Desire more imperative programming constructs?*  
Start by looking at the `etoolbox` package, and the `\foreach` command from `tikz`.
  - *Crave for an index?*  
Start with the `imakeidx` package
- If you however, want to understand  $\text{T}_{\text{E}}\text{X}$  in more detail, there are great resources
  - Overleaf itself offers good tutorials
  - The `TeXByTopic` book grants deep insights into the inner working of  $\text{T}_{\text{E}}\text{X}$
  - The original `T_{\text{E}}\text{X}book` is a great resource as well

# Defining your own macros

- For now, we have only used what  $\text{\LaTeX}$  and other packages provided us
- `\newcommand` however allows you to define your own commands!
- These commands can take numbered arguments and work similar to macro-replacements
  - `\newcommand[<number-of-arguments>]{<body>}`
  - You can refer to the arguments as #1, #2, #3, ...
- As basically everything is macro in  $\text{\LaTeX}$  the superficiality of this overview hurts the author mentally —  $\text{\LaTeX}$  is turing complete!

```

1 \documentclass{article}
2 \usepackage{xcolor}
3
4 \newcommand{\highlight}[1]{\textbf{#1led text}}
5
6 \begin{document}
7 This is a \highlight{highlight}!
8 \end{document}

```

This is a **highlighted text!**

# And macros?

Please inform yourself about how you can use macros to your advantage.  
The fun of  $\text{\LaTeX}$  starts with defining your own!

# Other $\text{\LaTeX}$ formats

- Besides  $\text{\LaTeX}$  (which is based on  $\text{\TeX}$ ) there are other formats
- While they are not as popular as  $\text{\LaTeX}$  they are in some ways more powerful
- $\text{\LuaTeX}$  offers Lua support and can load every font you have installed on your system
- If you are interested, read this [article](#).

# Source

- The complete source code for these slides can be found on GitHub at:  
<https://github.com/Eagleoutlce/kit-latex-intro>
- No penguins were harmed in the making of these slides.  
<https://github.com/Eagleoutlce/tikzpingus>
- QR-Codes have been created with `fancyqr!`



Source

•